Federated Learning for Private and Efficient Computation

Yaqin Si Bhaskar Ray

Abstract

The Federated Learning framework provides privacy-preserving algorithms to learn a valid machine learning model without centralizing the data. In the training process, a central server collects model parameter updates from clients and aggregates the local model updates to update the global model. Depending on the heterogeneity between clients and the trust level assigned to the clients, robust aggregation rules can be applied in the global model update procedure to protect the central server against attacks on the client side. On the other hand, clients borrow information from their neighbors in a decentralized Federated Learning framework to protect against the attack on the central server side. Centralized Federated Learning and Decentralized Federated Learning both incorporate massive client-server/client-client communication. Applying the quantization procedure to the model updates, increasing the local iterations before each communication round, and one-shot updating with higher-order approximation would save bandwidth in communication and enhance computation efficiency. We glimpse over some of the common set-ups for Federated Learning, starting at the most basic setup of the Federated Averaging Algorithm and considering extensions to suit various practical concerns, such as model poisoning due to attacks and heterogeneity between clients. We also present a simple simulation study to showcase the application of Federated Learning in logistic regression, demonstrating the behavior of estimates with respect to the sample size and the model size.

1 Motivation

Data is valuable and ubiquitous. Mobile devices keep track of the users' everyday activities and the interactions between users, and the healthcare provider's electronic health record (EHR) systems record every aspect of the wellness information for a large population. The information is so powerful that models trained on this type of data are so promising for further applications in business, governments, and academic research [1] [2]. But private data is also sensitive and vulnerable. Directly tracking user activity on mobile devices violates the user's privacy rights. Even with the user's permission, it is of great responsibility and risk to log user data onto the data center. As for the EHR data, centralizing the data not only violates privacy regulations, posing ethical and legal challenges, but it is also infeasible due to rapidly increasing data volume and technical limitations. Standard machine learning algorithms learn a model from centralized data, where data is exposed at different stages, as shown in figure 1. To protect data privacy and take advantage of big data, a method that can learn a valid model without collecting, distributing, or even looking at the data is in need. The Federated Learning methods provide a framework to train a model in a distributed manner while keeping the data locally instead of storing all data in the same data center.

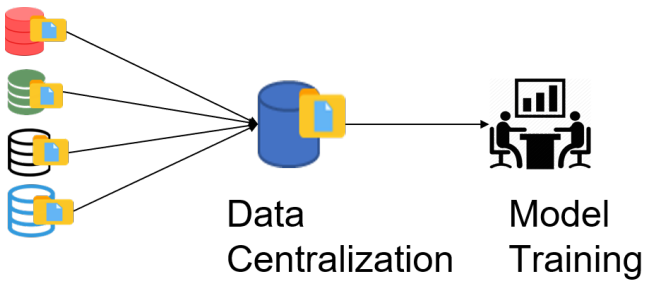


Figure 1: Data Flow in Standard Machine Learning

2 Terminology

In Federated Learning, denote the total number of `clients` as K, where a client (k) refers to a local data site with data D_k (for $k \in \{1, 2, \cdots, K\}$). A client can be a mobile device, an EHR system, or a local server with its own dataset. A client is also used as a local computation node to train a local model based on its data. `A central server` might have data (D_0) and trains a global model based on information from its data and the client's local data. The major advantage is that no raw data is shared across clients and the central server. In this report, $t = \{1, 2, \cdots, T\}$ is the number of communication rounds, and $r = \{1, 2, \cdots, R\}$ is the number of local training iterations. The parameter of interest is β. The estimate of the parameter at round t is represented as β_t, which could serve as model updates. A more general form of model updates is represented as g_t. In this report, g_t is the difference in estimating the parameters between two consecutive communication rounds or the gradient of the objective function.

3 The Federated Averaging (`FedAvg`) Algorithm

Federated learning provides a framework for training machine learning models without centralizing the data. This means decentralizing the training to each client whose outputs are aggregated to construct the final model.

The most intuitive setup involves a central server that communicates the parameters of interest with each client. Each client trains the model using the locally available data and returns the estimates (or updating scheme) to the server, which compiles the outputs.

As the data available to each client is different, a one-step result is nearly impossible except for trivial models. Therefore, we consider an iterative scheme that communicates the updated parameters (or updating scheme) back and forth between the central server and each client in multiple rounds. The Federated Averaging (`FedAvg`)[3] algorithm incorporates **Stochastic Gradient Descent (SGD)** in a distributed setup to carry out the training.

3.1 Mathematical Formulation

Consider a setup with K clients. Denote the data points present at client k by

$$D_k := \{x_{k,1}, \ldots, x_{k,n_k}\}$$

Let the pooled data be indexed as $x_1, \ldots, x_n$ where $n = \sum_{k=1}^{K} n_k$.
Assume a model with loss function $\ell(\cdot)$ and parameter of interest β. We wish to minimize the mean loss,

$$f(\beta) = \frac{1}{n}\sum_{i=1}^{n} \ell(x_i;\beta)$$

Observe, this is equivalently stated as minimizing

$$f(\beta) = \sum_{i=1}^{K} \frac{n_k}{n} F_k(\beta) \quad \text{and} \quad F_k(\beta) = \frac{1}{n_k}\sum_{i=1}^{n_k} \ell(x_{k,i};\beta)$$

where $F_k(\beta)$ is the mean loss at site k. Therefore, we can minimize the loss at each client based on the data there and aggregate them at the server under IID assumptions. Under IID assumption,

$$\mathbb{E}(F_k(\beta)) = f(\beta)$$

In the non-IID case, F_k could be a bad approximation of the overall mean loss function, f.

3.2 Algorithm

Algorithm 1 FedAvg Algorithm

Require: Number of clients, K; Client fraction in each round, C; Local mini-batch size, B; Number of local epochs, E and Learning rate, $\eta > 0$.

```
Server Executes:
Initialization: β_0 = random initial
for each round t = 1, 2, ... do                          ▷ Outer iteration for communication
    m ← max(C · K, 1)
    S_t ← (random set of m clients)
    for each client k ∈ S_t in parallel do               ▷ Inner iteration of server model
        β^k_{t+1} = ClientUpdate (k, β_t)
        β_{t+1} = Σ_{k=1}^{K} (n_k / n) β^k_{t+1}
    end for
end for
ClientUpdate (k, β) : // Run on client k
𝓑 ← ( split 𝓟_k into batches of size B)
for each local epoch i = 1, ..., E do
    for batch b ∈ 𝓑 do
        β ← w − η∇ℓ(β; b)                                ▷ Gradient averaged over mini-batch
    end for
end for
return β to server
```

Consider a fixed learning rate $\eta > 0$. At each step, each client uses a single iteration of SGD based on its data. Let, at iteration t, the value of the estimate be β_t. Denote,

$$g_k(\beta_t) = \nabla F_k(\beta_t)$$

Then, the update step can be formulated as

$$\beta_{t+1,k} = \beta_t - \eta g_k(\beta_t) \quad \text{[Client Update]}$$

$$\beta_{t+1} = \sum_{i=1}^{K} \frac{n_k}{n} \beta_{t+1,k} \quad \text{[Server Update]}$$

A major bottleneck in this algorithm is the communication cost between the server and the clients in each round. To avoid this, consider multiple rounds of iteration at each client before the result is communicated to the server. The IID assumption is important here as it guarantees the validity of each iteration based on the local estimate. As the mean loss function at the client is a "good" approximation of the overall mean loss function, the updates should result in "good" estimates of the true parameter locally.

It is possible to consider updates at each server step in C fraction of the clients. To begin with, we shall take $C = 1$, i.e., all clients are considered at each iteration. Denote E as the number of iterations at each client in each round. Further, we can consider the local training to be conducted in mini-batches of size B.

The initialization of the parameter is done globally. It has been observed in simulation studies[3] that global initialization produces a significant reduction in the loss on the total training set. It is also to be taken as a caution that general non-convex objective functions could lead to iterations converging in arbitrarily bad models.

3.3 Simulation Study

We conducted an empirical study to visualize the nature of the estimates produced over various considerations concerning the total number of data points and the number of clients. We considered a logistic regression model with univariate parameter β. Data was simulated as $X_i = (Z_i, Y_i)$ for $i = 1, \ldots, n$ where

$$Z_i \sim N(1,2) \text{ iid}$$

$$Y_i \sim Ber\left(\frac{1}{1+\exp[-\theta x]}\right) \text{ indep}$$

The true parameter was set as $\beta = 2$. The training was conducted over

- $n = 1000, 2000, \ldots, 10000$.
- For each n, $K = 10, 20, 50, 100$.

The hyperparameters were set as:

- $E = 1$.
- $n_k = n/K$ for $k = 1, \ldots, K$
- $B = \lfloor \log n_k \rfloor$

We observed the MSE of the estimates and the number of iterations till convergence in each case. Simulation results for some combinations will be presented in the near future due to the sheer amount of computational time required. In practice, however, a shared system would require much less time as the local updates are computed on local servers independently.

3.3.1 Results

The plots are provided below.

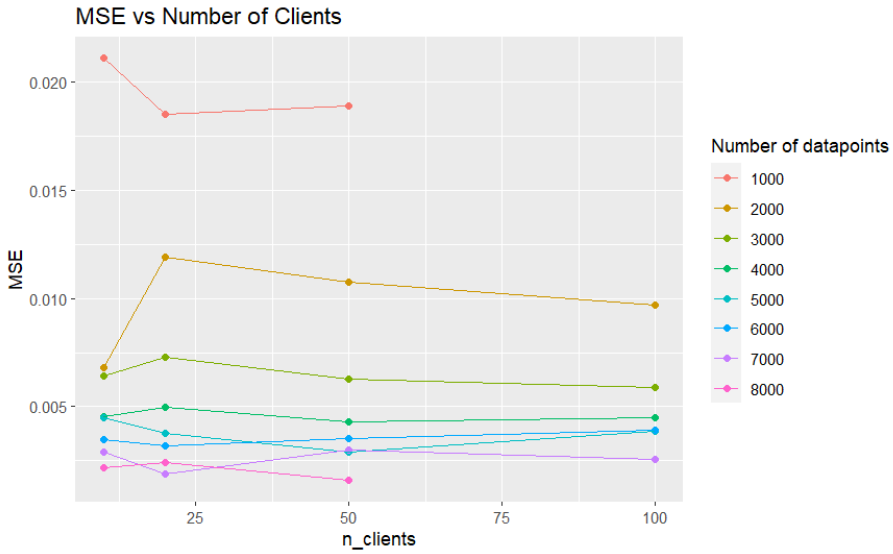


Figure 2: MSE plot

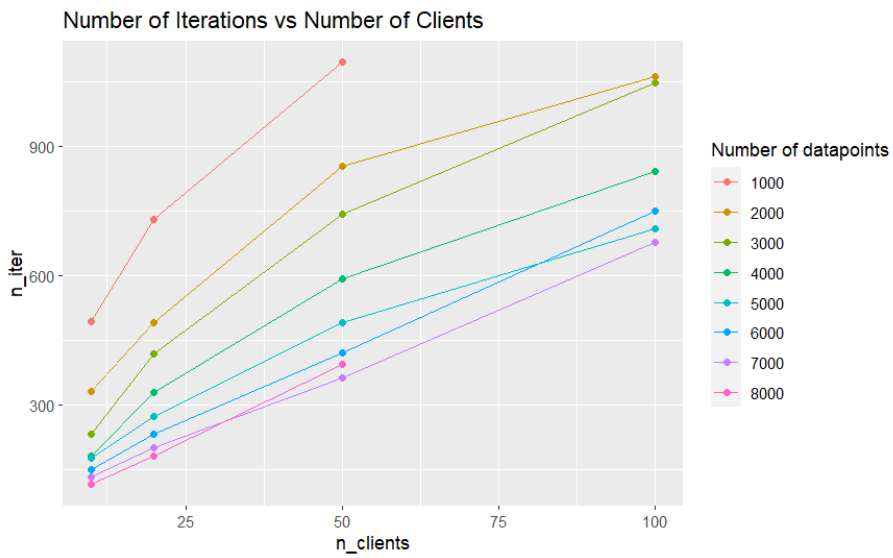


Figure 3: Number of Iterations plot

3.3.2 Observations

MSE values vary a little over changing the number of clients. This justifies that under IID assumptions, local updates closely approximate global updates. As the number of data points increases, MSE naturally drops.

With fewer number of datapoints at each client, the local estimates are expected to have higher MSE, and hence, we need more iterations to adjust for the fluctuations. This is evident from the plot of the

number of iterations required. At a fixed number of datapoints, the more the number of clients, the less the number of datapoints available to each client. Hence, the number of iterations increases.

4 Robust Federated Learning Dealing With Malicious Clients

The structure of the `FedAvg` algorithm that requires massive communication between every client and the central server makes it vulnerable to malicious clients. In an unhealthy situation, a bounded number of clients are poisoned, which means an attacker contaminates the local data or manipulates the local model updates. However, the central server has no prior information about which clients are malicious. In this situation, directly aggregating the local model updates without adjustments would poison the global model, leading to inaccurate predictions on the testing dataset. To deal with this problem, the central server could perform statistical analysis among the clients' local model updates before aggregating them to the global model update, excluding the suspicious ones and incorporating only those reliable ones. Some adjustments are labeled as Byzantine-robust aggregation rules, like the Krum method, the Trimmed Mean method, and the Median method. But, these methods do not change the fact that there is no root of truth in the original FedAvg aggregation rules and the Byzantine-robust aggregation rules. The Federated Learning framework is still vulnerable to those deliberately crafted attacks. The FLTrust[4] algorithm, in contrast, uses a small clean dataset as a surrogate of the ground truth, bootstraps a trust score for every client in each iteration, and applies the weighted average based on the trust score to update the global model.

4.1 Robust Federated Learning Aggregation Rules

- Coordinate-wise Aggregation

  Coordinate-wise aggregation rules collect the values of each parameter in all local model updates and process one parameter at a time. The global updates in each iteration use the Median or the Trimmed Mean of the local updates [5]. The Median method considers the median value of all local parameters, and the Trimmed Mean method requires a trim parameter $m < \frac{K}{2}$, removes the largest m values and the smallest m values, and computes the mean of $g_{t,k} \in \Gamma_{t,K-2m}$, where $\Gamma_{t,K-2m}$ represents the subset of the remaining $K-2m$ local model updates at communication round t.

$$g_t^d = \frac{1}{K-2m} \sum_{g_{t,k}^d \in \Gamma_{t,K-2m}} g_{t,k}^d$$

$$g_t = \text{Median}\{g_{t,k}\}$$

  The downside is that these methods suffer high information loss, where each global parameter update is based on only a small proportion of clients. If there is no attack, these methods will result in lower accuracy than the original `FedAvg` algorithm.

- Distance-based Rule

  The Krum method [6] is based on the Euclidean distance. In each iteration, the Krum method selects the local model updates with the smallest Euclidean-distance score as the global model

updates. Specifically, the Euclidean-distance score for client k is calculated as follows:

$$Score_{ED}(k) = \sum_{g_{t,i} \in \Gamma_{t,K-m-2}} \|g_{t,i} - g_{t,k}\|_2^2$$

where m is the upper limit of the number of malicious clients, $\Gamma_{t,K-m-2}$ is a subset of $K-m-2$ local model updates $g_{t,i}$ that has the smallest Euclidean distance to the model updates of client k, i.e., $g_{t,k}$. Generally, the Krum method compares pairwise distances and chooses the client that has the smallest sum of the distance between the client k and the other $K-m-2$ clients. Besides the common limitations mentioned in the coordinate-wise aggregation rules, the computation cost of the Krum method would become the bottleneck when K is large since the number of pairwise comparisons increases in a quadratic manner.

- Trust Score-based Rule The above aggregation rules consider client-client differences, but the Trust Score-based Rule incorporates "the root of truth". A higher trust score (S) is assigned to the local model updates that are more similar to the "truth", while a lower trust score is assigned to a client that deviates more from the "truth". The local model updates are normalized and weighted by the trust score when taking the weighted average. In this manner, all local data contributes to the global model with the trust score to control contribution levels.

$$g_t = \frac{1}{\sum_{k=1}^{K} S_k} \sum_{k=1}^{K} S_k g_{t,k}$$

4.2 FLTrust Algorithm

The `FLTrust` Algorithm [4] bootstraps trust score for the local model updates based on a server model generated and maintained from a small clean dataset. The central server updates the global model by considering both the server model update and the client's local model updates.

- Maintain Server Model

  The central server manually collects a clean training set called the root dataset. Then, it maintains a server model with the same procedure as the local model, meaning that the root dataset acts like a client and updates the server model following the same procedure discussed in the `FedAvg` Section.

- Bootstrap Trust Score

  As shown in figure 4, consider the model updates as a vector of dimension d, which has a direction in high-dimensional parameter space and a magnitude of scale. The `FLTrust` computes the Trust Score based on the direction, where we believe that, in each iteration, the global model update is around the direction of the server model update in the high dimensional parameter space. The server bootstraps the trust score for each client based on the similarity between the direction of the server model updates ($g_{t,0}$) and the direction of the clients' local model updates($g_{t,k}$, $k \in \{1, 2, \cdots, K\}$). The closer to the server model direction, the higher the trust score of a local model. Specifically, the central server calculates the ReLU-clipped Cosine Similarity between the server model updates and the clients' local model updates as the trust score. The Cosine Similarity measures the direction similarity between two vectors, and the ReLU function removes all negative impacts on a global model.

Algorithm 2 `FLTrust` Algorithm

Require: global learning rate $\alpha \geq 0$, local learning rate $\eta \geq 0$, communication round T

Initialization: $\beta_0 =$ random initial

for $t = 1, 2, ..., T$ **do** ▷ Outer iteration for communication

  for $r = \{1, 2, \cdots, R\}$ **do**

    Initialize $\beta_{t,0,0} = \beta_{t-1}$

    $\beta_{t,0,r} = \beta_{t,0,r-1} - \eta \nabla f_{(r-1)}$ and use $\beta_{t,0} = \beta_{t,0,R}$ as server model

    Server model updates $g_{t,0} = \beta_{t,0} - \beta_{t-1,0}$

  end for

  for $k = \{1, 2, \cdots, K\}$ **do**

    for $r = \{1, 2, \cdots, R\}$ **do**

      Initialize $\beta_{t,k,r} = \beta_{t-1}$

      $\beta_{t,k,r} = \beta_{t,k,r-1} - \eta \nabla f_{(r-1)}$ and use $\beta_{t,k} = \beta_{t,k,R}$ as local model

      Local model update $g_{t,k} = \beta_{t,k} - \beta_{t-1,k}$

    end for

    $S_k = ReLU(\frac{\langle g_{t,k}, g_{t,0} \rangle}{\|g_{t,k}\| \|g_{t,0}\|})$ ▷ Trust score

    $\tilde{g}_{t,k} = \frac{\|g_{t,0}\|_2}{\|g_{t,k}\|_2} g_{t,k}$ ▷ Normalize local model updates

  end for

  $g_t = \frac{1}{\sum_{k=1}^{N} S_k} \sum_{k=1}^{K} S_k \tilde{g}_{t,k}$

  Update global model as $\beta_t = \beta_{t-1} + \alpha g_t$

end for

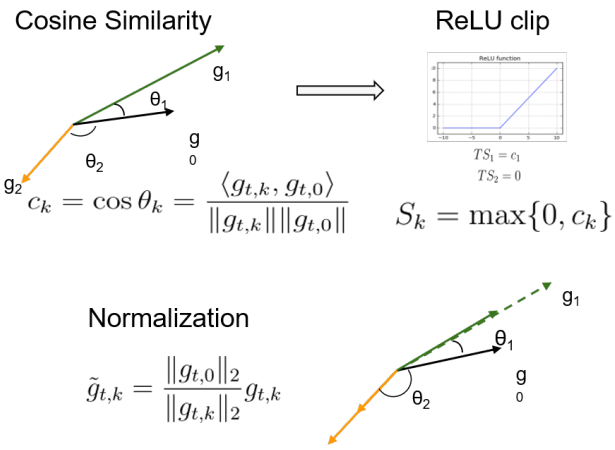


Figure 4: Processing Direction And Scale of The Model Updates

- Aggregation and Global Update

  Besides the direction of the local model updates, a malicious client can manipulate the scale of the local model update to dominate the global update. The `FLTrust` algorithm normalizes the local model updates ($\tilde{g}_t(k)$) to the same magnitudes such that they lie in the same hyper-sphere as the server model update, and the scale of the local model update does not dominate the global update. The aggregation generally follows the Truest Score-based aggregation rule as it averages the normalized local model according to the Trust Score and applies the update to the global model.

$$g_t = \frac{1}{\sum_{k=1}^{K} S_k} \sum_{k=1}^{K} S_k \tilde{g}_t(k)$$

4.3 Model Properties

The global optimum solution to the loss function is defined as $\beta^* = argmin_{\beta \in \mathscr{B}} f(D, \beta)$, where $f(D, \beta)$ is the loss function based on data D, and $\mathscr{B}$ is the parameter space. The FLTrust algorithm does not sacrifice the classification accuracy of the global model when there is no attack. In particular, under no attacks, the method would learn a global model that is as accurate as the global model learned by the FedAvg algorithm. In addition, the FLTrust algorithm does not increase the order of computation and communication compared to the FedAvg algorithm. What's more, the FLTrust algorithm learns an accurate global model in the presence of malicious clients within a limited error bound compared to the model of the FedAvg algorithm under no attacks.

It is shown that the difference between the global model learned by the FLTrust algorithm under attack ($\hat{\beta}$) and the optimal model (β^*) is bounded under assumptions 1 to 3.

`Assumption 1:` The loss funtion is $\mu-$ strongly convex and differentiable over the parameter space $\mathscr{B}$ with L-Lipshitz continuous gradient ∇f.

$$f(\hat{\beta}) \geq f(\beta) + \langle \nabla f(\beta), \hat{\beta} - \beta \rangle + \frac{\mu}{2} \|\beta - \hat{\beta}\|_2$$

$$\|\nabla f(\beta) - \nabla f(\hat{\beta})\|_2 \leq L \|\beta - \hat{\beta}\|_2,$$

where β and $\hat{\beta}$ are any vector in the parpmeter space $\mathscr{B}$. Furthermore, assume the loss function is $L1-$Lipschitz probabilistically, that is, for any $\delta \in (0,1)$, there exits L_1 such that:

$$Pr\left\{ \sup_{\beta, \hat{\beta} \in \mathscr{B}, \beta \neq \hat{\beta}} \frac{\|\nabla f(\beta) - \nabla f(\hat{\beta})\|_2}{\|\beta - \hat{\beta}\|_2} \leq L_1 \right\} \geq 1 - \frac{\delta}{3}$$

`Assumption 2:` The gradient of the loss function $\nabla f(\beta^*)$ at the optimal global model β^* is bounded. Moreover, the gradient difference $h(w) = \nabla f(\beta^*) - \nabla f(\beta)$ for any $\beta \in \mathscr{B}$ is bounded. Specifically, for any unit vector $u \in \mathscr{U}$, where $\mathscr{U}$ is the unit sphere $\mathscr{U} = \{u : \|u\|_2 = 1\}$ $\langle \nabla f(\beta^*), u \rangle$ is sub-exponential with σ_1 and γ_1, and $\frac{\langle h(w)(\beta) - E(h(w)(\beta)), u \rangle}{\|\beta - \beta*\|_2}$ is sub-exponential with σ_2 and γ_2. Specifically, define that for $\forall |\xi| \leq \frac{1}{\gamma_1}, \forall |\xi| \leq \frac{1}{\gamma_2}$,

$$\sup_{u \in \mathscr{U}} \mathbf{E}[\mathbf{exp}(\xi \langle \nabla \mathbf{f}(\beta^*), \mathbf{u} \rangle)] \leq \mathbf{e}^{\frac{\sigma_1^2 \xi^2}{2}}$$

$$\sup_{u \in \mathscr{U}} \mathbf{E}[\mathbf{exp}(\frac{\xi \langle \mathbf{h}(\mathbf{w})(\beta) - \mathbf{E}(\mathbf{h}(\mathbf{w})(\beta)), \mathbf{u} \rangle}{\|\beta - \beta^*\|_2})] \leq \mathbf{e}^{\frac{\sigma_1^2 \xi^2}{2}}$$

`Assumption 3:` Each client's local data and the central server's root data are sampled independently from the same distribution.

The error bound is expressed as $\lim_{t\to\infty}\|\beta-\hat{\beta}\|_2 \leq 12\alpha\frac{\triangle_1}{\rho}$, where $|1-\rho|<1$, α is the global learning rate, $\triangle_1$ and ρ are derived from assumption 1 and assumption 2. Specifically, we have

$$\rho = 1-(\sqrt{1-\frac{\mu^2}{4L^2}}+24\alpha\triangle_2+2\alpha L)$$
$$\triangle_1 = \sigma_1\sqrt{\frac{2}{|D_0|}}\sqrt{d\log 6+\log\frac{3}{\delta}}$$
$$\triangle_2 = \sigma_2\sqrt{\frac{2}{|D_0|}}\sqrt{d\log\frac{18L_2}{\sigma_2}+\frac{1}{2}d\log\frac{|D_0|}{d}+\log\frac{6\sigma_2^2 q\sqrt{|D_0|}}{\gamma_2\sigma_1\delta}}$$

where $|D_0|$ is the size of the root dataset, $L_2 = max\{L, L_1\}$, and q is a positive number that sets constraint to the parameter space such that $\|\beta-\beta^*\|_2 \leq q\sqrt{d}$ for any $\beta \in \mathscr{B}$.

In the Federated Learning framework, the central server has the global and local model updates for each iteration but knows nothing about which clients are malicious or the total number of malicious clients. Therefore, incorporating client-client / client-truth comparisons and applying different aggregation rules protect the central server against adversarial attacks.

5 Decentralized Federated Learning

So far, we have summarized the features of one type of Federated learning algorithm that has a central server to connect an enormous number of clients. To achieve privacy protection, the clients keep their data locally without sharing it with the central server. The clients receive the current model from the central server and update the model with the local data for multiple iterations, and the central server collects the local parameters and updates the global model according to its aggregation rules. Depending on the parameter heterogeneity between different clients and the level of trust that the central server assigns to the clients, the aggregation rules vary. The communication cost is high in Centralized Federated Learning since it requires massive communication between all clients and the central server in every aggregation round. Another concern of this framework is that attacking the central server will destroy the system, where all clients are connected to the central server.

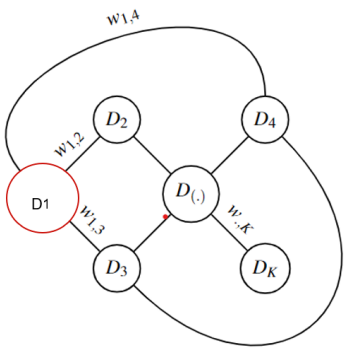
To deal with this situation, the Decentralized Federated Learning framework [7] is developed. The Decentralized Federated Learning does not incorporate a central server. Instead, the clients are connected as an undirected graph. The clients only communicate with the neighbors.

5.1 Decentralized FedAvg Algorithm

- **Mixing Matrix**

  The mixing matrix characterizes the communication relationship between clients. An undirected graph, denoted as $\mathscr{G} = (\mathscr{V}, \mathscr{E})$, is associated with the mixing matrix, where the vertex set $\mathscr{V} = \{1, \cdots, K\}$ represents a total number of K clients, and the edge set $\mathscr{E} \subseteq \mathscr{V} \times \mathscr{V}$ includes all communication links between nodes i and j. The associated mixing matrix W can be defined as

$w = [w_{i,j}] \in \mathbb{R}^{K \times K}$, where $w_{i,j} \geq 0$ for $i \neq j$ and $i, j \in \mathscr{V}$, which has the following properties: $null(I - W) = span\{\mathbf{1}\}$ and $I \succeq W \succeq -I$. For a graph with different adjacency matrices, the mixing matrix is not unique. The mixing matrix is symmetric $W = W^T$, that is, the matrix has real eigenvalues, and the eigenvalues can be ordered in the non-increasing order $\lambda_1(W) = 1 > \lambda_2(W) \geq \cdots \geq \lambda_K(W) > -1$. It can also serve as a probability transition matrix of a Markov chain, with $\lambda = max\{|\lambda_2(W)|, |\lambda_K(W)|\}$, which indicates the speed of the Markov chain converges to the stable state. The figure illustrates the graph relationship of K clients and the mixing matrix of D_1 with its neighbors.



	D_1	D_2	D_3	D_4
D_1	0	$w_{1,2}$	$w_{1,3}$	$w_{1,4}$
D_2	$w_{1,2}$	0	$w_{2,3}$	$w_{2,4}$
D_3	$w_{1,3}$	$w_{2,3}$	0	$w_{3,4}$
D_4	$w_{1,4}$	$w_{2,4}$	$w_{3,4}$	0

Figure 5: Communication Graph and Mixing Matrix

- **Decentralized FedAvg with Momentum**

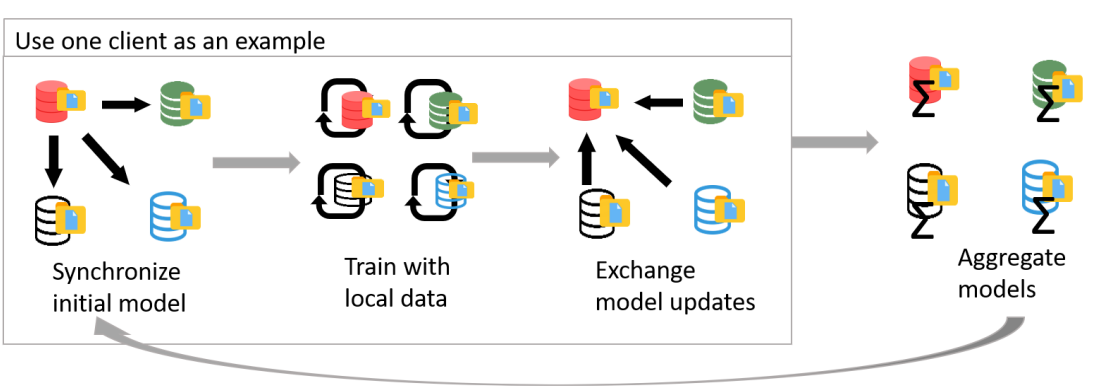


Figure 6: Decentralized FedAvg Algorithm

Consider a client (k) communicates with neighbor clients $i \in \mathscr{N}(k)$. The client k updates the estimation of parameters β_k based on its local dataset and information borrowed from the neighbors. The decentralized training process is demonstrated in Figure 6. Specifically,

Step 1, the model is initialized to all neighbors.

Step 2, all clients train the model based on local data for R iterations, where R is at least 1. This step is referred to as the inner iteration. For example, the inner iteration of client k starts from the current parameter estimation $\beta_{t-1,i}$, and updates it with R iterations of SGD with momentum, $\beta_{t,k,r} = \beta_{t,k,r-1} - \eta \nabla f_{k,r-1} + \theta(\beta_{t,k,r-1} - \beta_{t,k,r-2})$. At the end of this step, client k gets a local estimation of parameters for the current iteration t, $\beta_{t,k}$.

Step 3, neighbors return model updates to client k, denoted as $g_{t,i}$ for model updates from neighbor $i \in \mathcal{N}(k)$.

Step 4, client k aggregates the neighbor parameters as a weighted sum of its neighbors, $\tilde{\beta} = \sum_k w_{i,k}\beta_k$ and update its parameters as $\beta_{t+1,i} = \beta_{t,i} - \eta\nabla f_{t,i}$, where $\eta > 0$ is the learning rate.

Algorithm 3 Decentralized FedAvg with Momentum (DFedAvgM)

Require: Aggregation learning rate $\alpha \geq 0$, local learning rate $\eta \geq 0$, communication round T, inner iteration $R \geq 1$, momentum $0 \leq \theta < 1$

Initialization: $\beta^{(0)} = 0$

for $t = \{1, 2, ..., T\}$ **do** ▷ Outer iteration for communication

  for $k = \{1, 2, \cdots, K\}$ **do**

    for $r = \{1, 2, \cdots, R\}$ **do** ▷ Inner iteration for local update

      Initialize $\beta_{t,k,0} = \beta_{t,k,-1} = \beta_{t-1}$

      $\beta_{t,k,r} = \beta_{t,k,r-1} - \eta\nabla f_{k,r-1} + \theta(\beta_{t,k,r-1} - \beta_{t,k,r-2})$ and use $\beta_{t,k} = \beta_{t,k,R}$ as local model

      Send $g_{t,k} = \beta_{t,k} - \beta_{t-1,k}$ to neighbors $\mathcal{N}(k)$

    end for

    Model update of client k for $k \in \{1, 2, \cdots, K\}$ as $g_{t,k} = \sum_{i \in \mathcal{N}(k)} w_{i,k} g_{t,i}$

    Update client k for $k \in \{1, 2, \cdots, K\}$ as $\beta_{t,k} = \beta_{t-1,k} + \alpha g_{t,k}$

  end for

end for

5.2 Communication Efficient Decentralized Federated Learning

Compared to the Federated Learning Algorithm with a central server, Decentralized Federated Learning does not communicate between all clients and the central server, but the client-to-client communication can be costly, especially when the model is large or when the graph structure is complicated.

Communication between clients is indeed more time-consuming than local iterations. Performing multiple iterations of local updating before every communication round, as proposed in the above DFedAvgM algorithm, may benefit the performance in terms of computation time.

However, the volume of data transmitted between clients can also restrict the algorithm's efficiency since every client k needs to transfer the parameters to all of its neighbors $\mathcal{N}(k)$. When the graph gets more complicated and the number of neighbors grows, the data volume grows substantially.

A quantization method is applied to reduce the communication cost between clients.

- Quantization Function

  Given a constant $s > 0$ as a multiplier and an integer b that is limited by the bits of the computation system, the two numbers approximately represent a number v in the range of $v \in \{-2^{b-1}, 2^{b-1}\}$. Then we can find an integer v such that $vs \leq a < (v+1)s$ and use a stochastic procedure to replace a with a lower resolution, where $q(a) = \begin{cases} vs, w.p. 1 - \frac{a-ks}{s} \\ (v+1)s, w.p. \frac{a-ks}{s} \end{cases}$. The stochastic procedure has an unbiased expected value for any a.

  Applying the quantization function to the parameters $\beta = \{\beta_1, \beta_2, \cdots, \beta_d\} \in \mathbb{R}^d$ transmitted in

DFedAbgM algorithm, the multi-dimension procedure is defined as $Q(g) = \{q(g_1), q(g_2), \cdots, q(g_d)\}$ and satisfies $\mathbb{E}\|Q(\beta) - \beta\|^2 \leq \frac{s^2}{4}d$.

- Quantized DFedAvgM Algorithm

  The Quantized DFedAvgM algorithm has the same procedure as the DFedAvgM algorithm except sending quantized parameters $g_{t,k} = Q(\beta_{t,k} - \beta_{t-1,k})$ to the neighbors. It is straightforward to consider directly quantizing the parameters as $Q(\beta_{t,k})$. However, sometimes it is more feasible to quantize the difference of parameters as $Q(\beta_{t,k} - \beta_{t-1,k})$, like in the DNNs.

- Communication cost

  Consider a vector $\beta \in \mathbf{R}^d$ whose coordinates are all stored with 32 bits, which needs 32 bit/parameter $\times d$ parameter $\times$ Deg($\mathscr{N}(j)$) = $32d$Deg($\mathscr{N}(j)$) bits in client-client communication. The quantized algorithm with b limited bits, in contrast, needs $(32 + db) \times$ Deg($\mathscr{N}(j)$). Each client sents the pair $(s, Q^{(t)}(i)) \in R^{1+d}$, where s takes 32 bits, each of the q(t) takes b bits. When d is large, using $b < 32$ in the quantized method will significantly reduce the communication cost.

5.3 Convergence Properties

- Assumptions

  `Assumption 1:` The function f_l is differentiable and ∇f_l is L-Lipschitz continuous, that is, $\|\nabla f(\beta) - \nabla f(\hat{\beta})\|_2 \leq L\|\beta - \hat{\beta}\|_2$, where β and $\hat{\beta}$ are any vector in the parpmeter space $\mathscr{B}$.
  `Assumption 2:` The gradient function ∇f_l have σ_l bounded local variance, that is $\mathbf{E}\|\nabla F_l(\beta, \mathscr{D}) - \nabla f_l(\beta, D_l)\|_2^2 \leq \sigma_l^2$ for all clients. Moreover, the total variance is also bounded, that is, $\frac{1}{n}\sum_{l=1}^{n}\|\nabla f_l(\beta) - \nabla f(\beta)\|_2^2 \leq \sigma_g^2$. A positive σ_g^2 indicate the heterogeneity of local client datasets, and if clients have IID datasets, $\sigma_g^2 = 0$.
  `Assumption 3:` $\|\nabla f_l\|_2 < C$ for some $C > 0$

- Convergence of DFedAvgM for General Nonconvex objective function

  In addition to assumptions 1 to 3, assume the learning rate satisfies that $0 < \eta \leq \frac{1}{8LR}$ and $64L^2R^2\eta^2 + 64LR\eta < 1$. Let $\beta^{(t)}(l)$ represent the sequence generated by DFedAbgM for client l. Considering the average point of the K clients defined as $\bar{\beta}^t = \frac{1}{K}\sum_{l=1}^{K}\beta^t(l)$, we have

$$\min_{0 \leq t \leq T} \mathbf{E}\|\nabla f(\bar{\beta}^t)\|_2^2 \leq \frac{2f(\bar{\beta}^t) - 2\min f}{\psi_1(R,\eta)T} + \psi_2(R,\eta) + \psi_3(R,\eta),$$

  where ψ_1, ψ_2, and ψ_3 are functions of R and η. Then, choose $\eta = \Theta(\frac{1}{LR\sqrt{T}})$, we have an explicit rate on T following the above inequality. As the communication round T is large enough, the rate holds that $\bar{\beta}^t = \frac{1}{n}\sum_{l=1}^{n}\beta^t(l)$, we have

$$\begin{aligned}\min_{0 \leq t \leq T} \mathbf{E}\|\nabla f(\bar{\beta}^t)\|_2^2 \leq \mathscr{O}(&\frac{(1-\theta)(f(\bar{\beta}^t) - \min f)}{\sqrt{T}} \\ &+ \frac{(1-\theta)(\sigma_l^2 + R\sigma_g^2) + \frac{\theta^2}{(1-\theta)}R(\sigma_l^2 + C^2)}{R\sqrt{T}} \\ &+ \frac{(1-\theta)(\sigma_l^2 + R\sigma_g^2 + RC^2) + \frac{\theta^2}{(1-\theta)}R(\sigma_l^2 + C^2)}{(1-\lambda)RT^{\frac{3}{2}}}),\end{aligned}$$

where $0 < \lambda < 1$ is associated with the convergence rate of the mixing matrix converging to the stable state. We can see that the speed of DFedAvgM can be improved by increasing the inner iteration R. To reach any given $\varepsilon > 0$, the DFedAvgM algorithm needs $T = \mathscr{O}(\frac{1}{\varepsilon^2})$ communication rounds. The momentum can accelerate the algorithm when $f(\bar{\beta}^t) - \min f) + \sigma_g^2 >> \sigma_l^2 + C^2$, large θ will improve the convergence rate, but if $f(\bar{\beta}^t) - \min f) + \sigma_g^2 << \sigma_l^2 + C^2$, the performance gets worse with a larger momentum.

- Convergence of DFedAvgM with Quantization Under the same assumptions 1 to 3 and the same learning rate as the DFedAbgM algorithm $\eta = \Theta(\frac{1}{LR\sqrt{T}})$, for a sequence $\beta^{(t)}(l)$ generated from DFedAvgM with Quantization algorithm, it holds that

$$\begin{aligned}\min_{0\le t\le T} \mathbf{E}\|\nabla f(\bar{\beta}^t)\|_2^2 \le \mathscr{O}(&\frac{(1-\theta)(f(\bar{\beta}^t) - \min f)}{\sqrt{T}} \\ &+ \frac{(1-\theta)(\sigma_l^2 + R\sigma_g^2) + \frac{\theta^2}{(1-\theta)}R(\sigma_l^2 + C^2)}{R\sqrt{T}} \\ &+ \frac{(1-\theta)(\sigma_l^2 + R\sigma_g^2 + RC^2) + \frac{\theta^2}{(1-\theta)}R(\sigma_l^2 + C^2)}{(1-\lambda)RT^{\frac{3}{2}}} + \sqrt{T}s)\end{aligned}$$

To reach any given $\varepsilon > 0$, the DFedAvgM with quantization algorithmn also needs $T = \mathscr{O}(\frac{1}{\varepsilon^2})$ communication rounds. However, if the desired error is set at

$$\varepsilon > (1-\theta)\sqrt{3LCs}d^{\frac{1}{4}}\sqrt{2(f(\bar{\beta}^t) - \min f) + \frac{8\sigma_l^2}{R} + 32\sigma_g^2 + \frac{64\theta^2(\sigma_l^2 + C^2)}{(1-\theta)^2}},$$

and $b < \frac{128}{9} + \frac{32}{d}$, the quantization algorithm beats the original DFedAvgM, and the error bound decreases as the number of inner iterations R increase, which indicates that multiple inner iterations are required.

In the DFedAvgM algorithm, communication is limited to neighbors. A Decentralized Federated Averaging with momentum (DFedAvgM) algorithm is developed for integrating clients' information without a central server, which provides an option when local clients have concerns about the central server's security. Moreover, when the number of clients is large and the communication becomes complicated, quantization methods saves communication cost in terms of the data volume transmitted and communication rounds required to reach some non-zero error bound. In addition, under the general non-convexity assumption, the non-zero error bound can be reduced by increasing the number of training iterations within the local clients before communicating with neighbors.

6 Federated Learning with Heterogenous data

The methods considered are based on data obtained from the same distribution with a common parameter. This is seldom the case in real life. Various subpopulations and environments can cause heterogeneity in the distribution of the data.

When considering a Federated Learning approach, we are interested in a common parameter determining the distribution at all sites. Duan, Ning, Chen[8] consider an extension of the surrogate likelihood

approach[9][10], considering site specific nuisance parameters γ_j and a common global parameter β. The data is considered to be distributed as

$$X_{ij} \sim f(x;\beta,\gamma_j); \quad j = 1,\ldots,K$$

Denote, $\Gamma := \{\gamma_1,\ldots,\gamma_K\}$. The complete data log-likelihood can then be represented as

$$L(\beta,\Gamma) = \frac{1}{Kn}\sum_{j=1}^{K}\sum_{i=1}^{n}\log f\left(X_{ij};\beta,\gamma_j\right) := \frac{1}{K}\sum_{j=1}^{K} L_j(\beta,\gamma_j)$$

where $L_j(\beta,\gamma_j) = \sum_{i=1}^{n}\log f\left(X_{ij};\beta,\gamma_j\right)$ is the log likelihood function at each client.

6.1 Semiparametric Approach

In the above set-up, for the purpose of federated learning, we are not interested in the nuisance parameter. Duan, Ning, Chen[8] have considered using the efficient score function for distributed inference on such heterogeneous data, treating it as a semiparametric set-up. This reduces the influence of the nuisance parameter, Γ, on the inference and protects against less accuracy of estimates of Γ. The contribution to the efficient score function due to the nuisance parameter from the j-th client, $s_j\left(y;\beta,\gamma_j\right)$ is a projection of the score function of β on the orthogonal space of the score function of γ_j[11].

$$s_j\left(y;\beta,\gamma_j\right) = \nabla_\beta \log f\left(y;\beta,\gamma_j\right) - I_{\beta\gamma}^{(j)} I_{\gamma\gamma}^{(j)^{-1}} \nabla_\gamma \log f\left(y;\beta,\gamma_j\right)$$

where $I_{\gamma\gamma}^{(j)}$ and $I_{\gamma\beta}^{(j)}$ are the corresponding submatrices of the information matrix for the j th client, $I^{(j)} = \mathbb{E}\left\{-\nabla^2 L_j\left(\theta_j^*\right)\right\}$. $s_j\left(y;\beta,\gamma_j\right)$ satisfies the following properties:

- The asymptotic variance of the estimator of β obtained from solving the efficient score function attains the Cramér-Rao lower bound and is, thus, an efficient estimator.
- $\mathbb{E}\left\{\nabla_\gamma s_j\left(y;\beta,\gamma_j\right)\right\} = 0$. This implies that the efficient score function is robust against γ_j.

Aggregating the contributions from all clients, the combined efficient score function can be defined as

$$S(\beta,\Gamma) = \frac{1}{Kn}\sum_{j=1}^{K}\sum_{i=1}^{n} s_j\left(y_{ij};\beta,\gamma_j\right)$$

6.2 Surrogate Likelihood

Federated learning aims to approximate the above efficient score function using transmitted data from each site. Duan, Ning, Chen[8] construct a surrogate efficient score equation for this purpose that expresses the expectation of the function in terms of data from the first client. Denoting the true parameters by β^* and γ_j^* for $j = 1,\ldots,K$, a function $g^*(y;\beta)$ is constructed such that

$$\mathbb{E}_{\theta_1^*}\left\{\nabla_\beta^k g^*\left(Y_{i1};\beta\right)\right\} = \mathbb{E}\left\{\nabla_\beta^k S\left(\beta,\Gamma^*\right)\right\} = \frac{1}{K}\sum_{j=1}^{K}\mathbb{E}_{\theta_j^*}\left\{\nabla_\beta^k s_j\left(Y_{ij};\beta,\gamma_j^*\right)\right\} \quad \forall k \geq 1$$

where $\mathbb{E}_{\theta_j^*}(\cdot)$ denotes the expectation concerning the distribution $f\left(y,\beta^*,\gamma_j^*\right)$, and $\mathbb{E}(\cdot)$ denotes the expectation with respect to the joint distribution of the full data.

6.3 Density Ratio Tilting Method

Observe, in the above expressions, $g^*(Y_{i1};\beta)$ depends on $Y_{i1} \sim f(y,\beta^*,\gamma_1^*) \not\equiv f\left(y,\beta^*,\gamma_j^*\right)$ for $j \neq 1$. Thus, Duan, Ning, Chen[8] propose to construct $g^*(y;\beta)$ by using the density ratio tilting method as

$$g^*(y;\beta) = \frac{1}{K}\sum_{j=1}^{K} \frac{f\left(y;\beta^*,\gamma_j^*\right)}{f(y;\beta^*,\gamma_1^*)} s_j\left(y_{ij};\beta,\gamma_j^*\right),$$

where the density ratio $\frac{f\left(y;\beta^*,\gamma_j^*\right)}{f\left(y;\beta^*,\gamma_1^*\right)}$ accounts for the heterogeneity of the distributions. They show that $\mathbb{E}_{\theta_1^*}\left\{\nabla_\beta^k g^*(Y_{i1};\beta)\right\} = \mathbb{E}\left\{\nabla_\beta^k S(\beta,\Gamma^*)\right\}$ for any $k \geqslant 0$ and observation Y_{i1} at the first client.

6.4 Expressing as Federated Learning Set-up

As $g^*(y;\beta)$ depends on unknown values of the true parameters, they propose using a natural surrogate using some initial estimators, $\bar{\beta}$ and $\bar{\gamma}_j$ of the parameters and the density ratio tilting estimator of $I^{(j)}$,

$$\tilde{H}^{(1,j)} = -\frac{1}{n}\sum_{i=1}^{n} \nabla^2 \log f\left(y_{i1};\bar{\beta},\bar{\gamma}_j\right) \frac{f\left(y_{i1};\bar{\beta},\bar{\gamma}_j\right)}{f\left(y_{i1};\bar{\beta},\bar{\gamma}_1\right)}.$$

for $j = 1,\ldots,K$. Then, writing in terms of the estimators,

$$g(y;\beta,\bar{\beta},\bar{\Gamma}) = \frac{1}{K}\sum_{j=1}^{K}\left[\frac{f\left(y;\bar{\beta},\bar{\gamma}_j\right)}{f\left(y;\bar{\beta},\bar{\gamma}_1\right)}\left\{\nabla_\beta \log f(y;\beta,\bar{\gamma}_j) - \tilde{H}_{\beta\gamma}^{(1,j)}\left\{\tilde{H}_{\gamma\gamma}^{(1,j)}\right\}^{-1}\nabla_\gamma \log f(y;\beta,\bar{\gamma}_j)\right\}\right].$$

Denoting $U_1(\beta;\bar{\beta},\bar{\Gamma}) = \frac{1}{n}\sum_{i=1}^{n} g\left(y_{i1};\beta,\bar{\beta},\bar{\Gamma}\right)$, the surrogate efficient score function can be defined as

$$\tilde{U}(\beta;\bar{\beta},\bar{\Gamma}) = U_1(\beta;\bar{\beta},\bar{\Gamma}) + \frac{1}{K}\sum_{j=1}^{K}\left\{\nabla_\beta L_j\left(\bar{\beta},\bar{\gamma}_j\right) - \bar{H}_{\beta\gamma}^{(j)}\left(\bar{H}_{\gamma\gamma}^{(j)}\right)^{-1}\nabla_\gamma L_j\left(\bar{\beta},\bar{\gamma}_j\right)\right\} - U_1(\bar{\beta};\bar{\beta},\bar{\Gamma}),$$

where $\bar{H}_{\beta\gamma}^{(j)} = \nabla_{\beta\gamma} L_j\left(\bar{\beta},\overline{\gamma_j}\right)$ and $\bar{H}_{\gamma\gamma}^{(j)} = \nabla_{\gamma\gamma} L_j\left(\bar{\beta},\bar{\gamma}_j\right)$.

Note, in this set-up, it is only required to transfer the score vector $S_j\left(\bar{\beta},\bar{\gamma}_j\right) = \nabla_\beta L_j\left(\bar{\beta},\bar{\gamma}_j\right) - \bar{H}_{\beta\gamma}^{(j)}\left(\bar{H}_{\gamma\gamma}^{(j)}\right)^{-1}\nabla_\gamma L_j\left(\bar{\beta},\bar{\gamma}_j\right)$ from each client and some initial estimators. The surrogate efficient score estimator $\tilde{\beta}$ is obtained by solving the following for β at client 1.

$$\tilde{U}(\beta;\bar{\beta},\bar{\Gamma}) = 0.$$

6.4.1 Removing dependence on specific client

Here, the aggregation is dependent on client 1. There could be two ways of handling this.

- Choose client 1 as the central server. This gives rise to a centralized framework.
- Assign this task in parallel to all clients. This does not require a central server and can be made robust to attacks on specific clients.

We shall consider the latter as it leads to a more generalized and flexible design.

At the j-th client, define client-specific surrogate score function, $\tilde{U}_j(\beta;\bar{\beta},\bar{\Gamma})$ as above. Equating them to 0, from the solutions, $\tilde{\beta}_j$'s, we can calculate the overall output as

$$\tilde{\beta}_{\text{all}} = \frac{1}{K}\sum_{j=1}^{K}\tilde{\beta}_j$$

6.5 Convergence

Under certain regularity conditions stated in [8], it can be shown that if the number of iterations $T \geqslant 2$, then

$$\mathbb{E}\left\|\tilde{\beta}^{(T)} - \hat{\beta}\right\|_2 \leqslant \frac{C_1}{(K)^{1/2}n} + \frac{C_2}{n^{3/2}},$$

where C_1 and C_2 are positive constants not related to n and K and $\hat{\beta}$ is the global estimator (without a federated learning approach).

7 Future Direction

Further studies can encompass extensions of the parametric setups and more generalized models. The current framework considers client-client communication and server-client communication, we can certainly extend this framework to more involved structures, like including hierarchical topology in the communication rounds and embracing the graph theory into the Federated Learning framework.
Protecting the system from attacks is another interesting topic. Further considerations in this regard can be leveraging the decentralized Federated Learning framework to protect against attacks on the central server and formally establishing its efficiency and safety properties. More generally, to test the model properties, a set of standard attacks and performance criteria is in need.
Moreover, there is a practical need to consider unbalanced data and situations with missing data. The prospects are immense and diverse in theoretical and practical studies of the various models of Federated Learning.

References

1. Rieke, N. *et al.* The future of digital health with federated learning. *NPJ digital medicine* **3,** 1–7 (2020).
2. Konečnỳ, J., McMahan, H. B., Ramage, D. & Richtárik, P. Federated optimization: Distributed machine learning for on-device intelligence. *arXiv preprint arXiv:1610.02527* (2016).
3. McMahan, B., Moore, E., Ramage, D., Hampson, S. & Arcas, B. A. y. *Communication-Efficient Learning of Deep Networks from Decentralized Data* in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics* (eds Singh, A. & Zhu, J.) **54** (PMLR, 20–22 Apr 2017), 1273–1282. `https://proceedings.mlr.press/v54/mcmahan17a.html`.
4. Cao, X., Fang, M., Liu, J. & Gong, N. Z. Fltrust: Byzantine-robust federated learning via trust bootstrapping. *arXiv preprint arXiv:2012.13995* (2020).
5. Yin, D., Chen, Y., Kannan, R. & Bartlett, P. *Byzantine-robust distributed learning: Towards optimal statistical rates* in *International Conference on Machine Learning* (2018), 5650–5659.
6. Blanchard, P., El Mhamdi, E. M., Guerraoui, R. & Stainer, J. Machine learning with adversaries: Byzantine tolerant gradient descent. *Advances in Neural Information Processing Systems* **30** (2017).
7. Sun, T., Li, D. & Wang, B. Decentralized federated averaging. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2022).
8. Duan, R., Ning, Y. & Chen, Y. Heterogeneity-aware and communication-efficient distributed statistical inference. *Biometrika* **109,** 67–83. ISSN: 1464-3510. eprint: `https://academic.oup.com/biomet/article-pdf/109/1/67/42362094/asab007.pdf`. `https://doi.org/10.1093/biomet/asab007` (Feb. 2021).
9. Wang, J., Kolar, M., Srebro, N. & Zhang, T. *Efficient Distributed Learning with Sparsity* in *Proceedings of the 34th International Conference on Machine Learning* (eds Precup, D. & Teh, Y. W.) **70** (PMLR, June 2017), 3636–3645. `https://proceedings.mlr.press/v70/wang17f.html`.
10. Jordan, M. I., Lee, J. D. & Yang, Y. Communication-Efficient Distributed Statistical Inference. *Journal of the American Statistical Association* **114,** 668–681. eprint: `https://doi.org/10.1080/01621459.2018.1429274`. `https://doi.org/10.1080/01621459.2018.1429274` (2019).
11. Vaart, A. W. v. d. *Asymptotic Statistics* (Cambridge University Press, 1998).

Appendix

A Simulation Codes

```
# Packages

library("foreach")
library("doParallel")
library("tidyverse")
############################

# Client Executable

client_update <- function(w, grad_l, b, lr, dat)
{
  b.num = floor(nrow(dat)/b) - 1

  for(batch in 0:(b.num - 1))
  {
    subdat = dat[(batch*b + 1):((batch + 1)*b), ]
    print(subdat)
    w = w - lr*mean(apply(subdat, MARGIN = 1, FUN = grad_l, w))
  }

  subdat = dat[((batch + 1)*b + 1):nrow(dat), ]
  w = w - lr*mean(apply(subdat, MARGIN = 1, FUN = grad_l, w))

  return(w)
}

############################

# Server Aggregation

server_agg <- function(grad_l, n_clients = 2, lr = 0.1, dat, w_
   start, tol = 1e-05)
{
  client.size = nrow(dat)/n_clients
  flag = 1
  niter = 0

  client_update <- function(w_client, grad_l, b, lr, dat2)
```

```
  {
    b.num = floor(nrow(dat2)/b) - 1

    for(batch in 0:(b.num - 1))
    {
      subdat = dat2[(batch*b + 1):((batch + 1)*b), ]
      w_client = w_client - lr*mean(apply(subdat, MARGIN = 1, FUN
    = grad_l, w_client))
    }

    subdat = dat[((batch + 1)*b + 1):nrow(dat), ]
    w_client = w_client - lr*mean(apply(subdat, MARGIN = 1, FUN =
    grad_l, w_client))

    return(w_client)
  }

  while(flag == 1)
  {
    niter = niter + 1
    w_update <- foreach(client = 1:n_clients, .combine = sum)%
   dopar%
    {
      client_update(w_start, grad_l, floor(log(client.size)), lr,
    dat[((client - 1)*client.size + 1):(client*client.size), ])
    }

    w_update = w_update/n_clients

    if(abs(w_update - w_start) <= tol)
      flag = 0

    w_start = w_update
  }
  return(c(w_update, niter))
}

##############################

# Generating Data

expit = function(x){
```

```
  1/(exp(-x)+1)
}

grad_l_log_reg <- function(x, theta)
{
  return(x[2]*(-x[1] + 1/(exp(-theta*x[2])+1)))
}

gen_dat_log_reg <- function(N, theta)
{
  X = rnorm(N, 1, 2)
  Y = rbinom(N, 1, expit(theta*X))

  return(cbind(Y, X))
}

##############################

# Running Simulations

N_list = seq(1000, 10000, by = 1000)

theta_0 = 2

stats_list = NULL

cl = makeCluster(10)
registerDoParallel(cl)

for(N in N_list)
{
  print(N)

  for(n_clients in c(10, 20, 50, 100))
  {
    print(n_clients)
    est_iter_vec <- foreach(iter = 1:100, .combine = rbind, .
   packages = c("foreach", "doParallel"))%dopar%
    {
      X = gen_dat_log_reg(N, theta_0)
      as.vector(server_agg(grad_l = grad_l_log_reg, n_clients,
   dat = X, w_start = 1, lr = 0.01))
```

```
    }
    MSE = mean((est_iter_vec[, 1] - theta_0)^2)
    bias = mean((est_iter_vec[, 1] - theta_0))
    vars = var(est_iter_vec[, 1])

    n_iter = mean(est_iter_vec[, 2])

    stats_list = rbind(stats_list, c(N, n_clients, MSE, bias,
   vars, n_iter))
  }
}

stopCluster(cl)

##############################

# Storage (Saving to file done to facilitate distributed running)

dat.prev = read.csv("FedAvgsim.csv", header = T)[, -1]
write.csv(as.data.frame(rbind(dat.prev, stats_list)), "FedAvgsim.
   csv")

##############################

# Plots

dat_to_plot = as.data.frame(read.csv("FedAvgsim.csv", header = T)
   [, -1])
colnames(dat_to_plot) <- c("N", "n_clients", "MSE", "Bias", "
   Variance", "n_iter")

# MSE Plot

plt_mse <- ggplot(data = dat_to_plot, mapping = aes(x = n_clients
   , y = MSE, group = as.factor(N))) +
          geom_point(aes(color = as.factor(N))) +
          geom_line(aes(color = as.factor(N))) +
          labs(title = "MSE␣vs␣Number␣of␣Clients") +
          scale_color_discrete("Number␣of␣datapoints")

print(plt_mse)
```

```
# Iterations Plot

plt_iter <- ggplot(data = dat_to_plot, mapping = aes(x = n_
   clients, y = n_iter, group = as.factor(N))) +
          geom_point(aes(color = as.factor(N))) +
          geom_line(aes(color = as.factor(N))) +
          labs(title = "Number␣of␣Iterations␣vs␣Number␣of␣Clients
   ") +
          scale_color_discrete("Number␣of␣datapoints")

print(plt_iter)
```